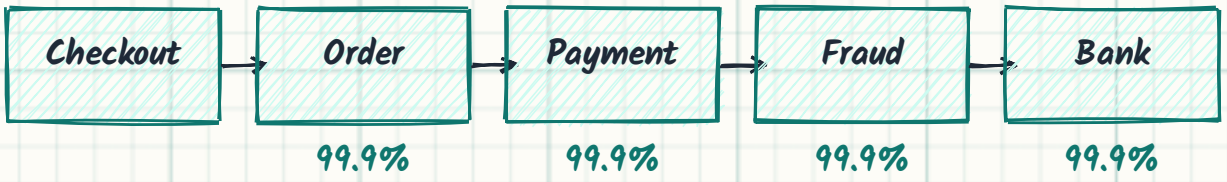
REST vs gRPC

Lesson 5 · synchronous communication between services

In this lesson → ① The call-chain problem ② Sync vs async ③ REST ④ gRPC ⑤ REST vs gRPC ⑥ REST outside, gRPC inside ⑦ Dangerous chains ⑧ 5 rules ⑨ Interview Q&A
Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① The call-chain problem



0.999 × 0.999 × 0.999 × 0.999 ≈ 99.6%

latency adds up: 50 + 80 + 120 + 200 ms

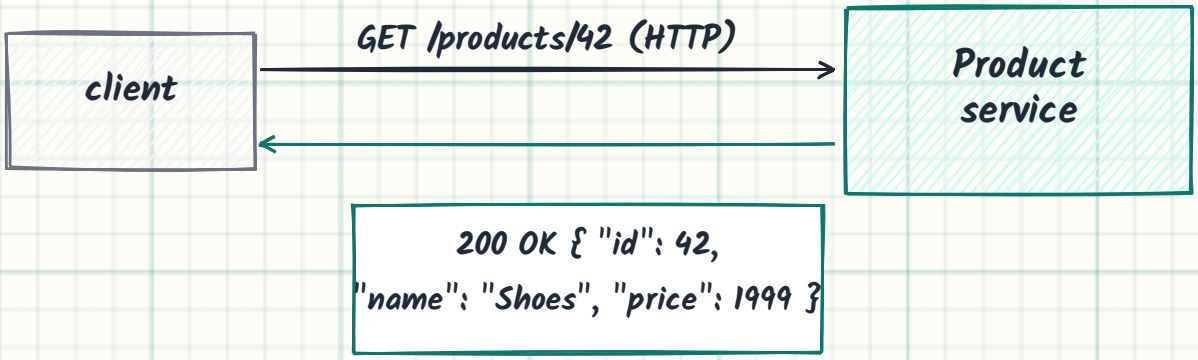
> **Availability of a chain** = multiply the availability of each hop: 0.999 × 0.999 × 0.999 × 0.999 ≈ **99.6%**. Latency **adds**: 50 + 80 + 120 + 200 ms.

② Sync vs async

	Synchronous	Asynchronous
How	request → wait → response	send message → move on
Like	a phone call	a text message
Tech	REST, gRPC	events, queues (Lesson 6)
Good	simple, instant answer	decoupled, absorbs spikes
Risk	caller blocked, both must be up	eventual consistency, harder to trace

③ REST

> **REST** = resources with URLs + HTTP verbs + (usually) JSON. Stateless requests with status codes.



Verb	Meaning	ShopNest example
GET	read	GET /products/42
POST	create	POST /orders
PUT / PATCH	update	PATCH /orders/7 (status)
DELETE	remove	DELETE /cart/items/3

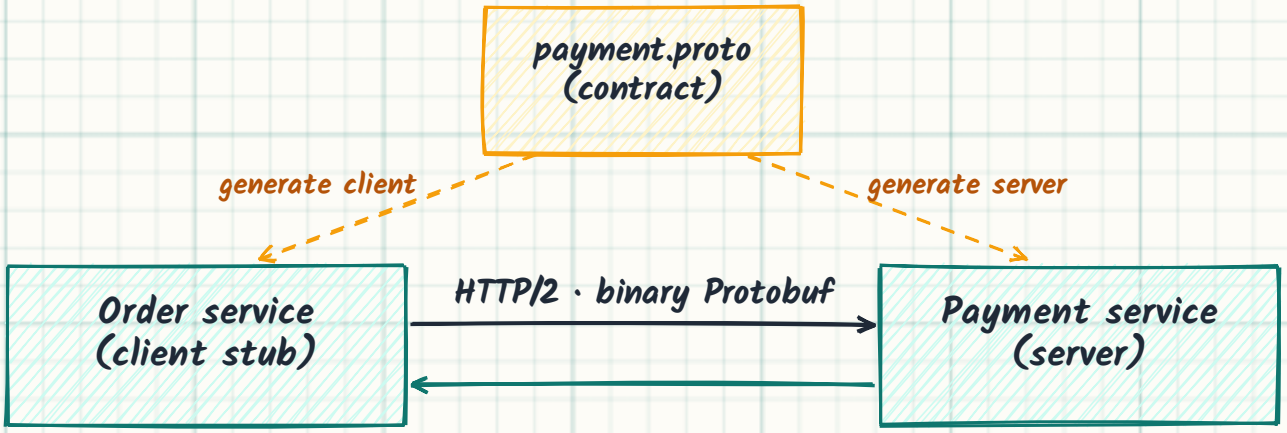
- ✓ Works everywhere
- ✓ Human readable, easy to debug
- ✓ HTTP caching
- ✗ Bigger JSON payloads
- ✗ No strict contract (use OpenAPI)
- ✗ Per-request overhead

Calling REST from Spring Boot

```
RestClient inventory = RestClient.builder()
    .baseUrl("http://inventory-service")
    .build();                  // set timeouts via a request factory
StockDto stock = inventory.get()
    .uri("/stock/{id}", productId)
    .retrieve()
    .body(StockDto.class);
```

④ gRPC

> **gRPC** (open-sourced by Google, 2015) = contract-first RPC: a .proto file in **Protocol Buffers** generates client & server code; binary messages over `HTTP/2`.



0 1 0 1 1 0 small & fast

```
syntax = "proto3";
service PaymentService {
  rpc Charge (ChargeRequest) returns (ChargeResponse);
}
message ChargeRequest {
  string order_id     = 1;
  int64 amount_paise = 2;  // ₹19.99 → 1999
  string currency     = 3;
}
message ChargeResponse {
  string payment_id = 1;
  bool   success    = 2;
}
```

Field numbers
Numbers (not names) go on the wire → small messages, and you can **add fields** without breaking old clients.

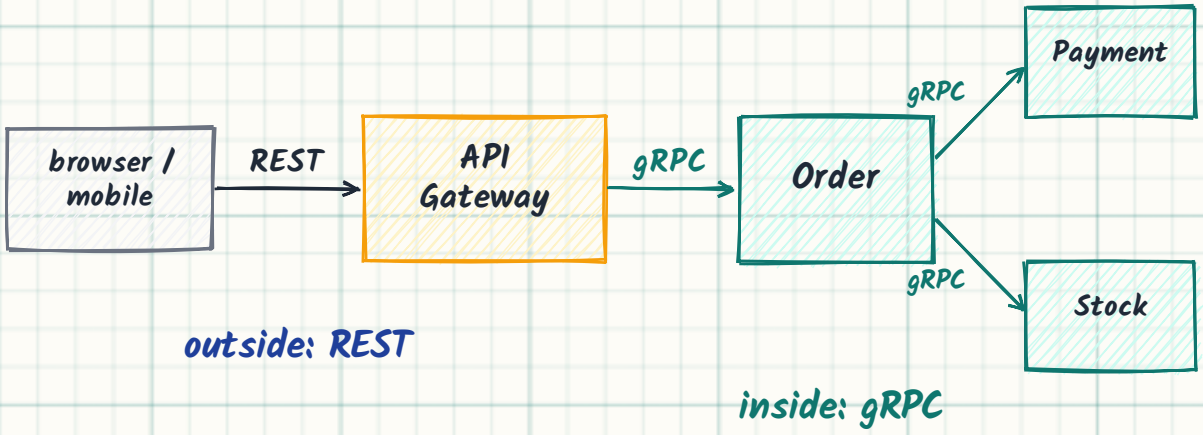
→ **4 call types**: unary, server streaming, client streaming, bidirectional

→ **Deadlines** built in

5 REST vs gRPC

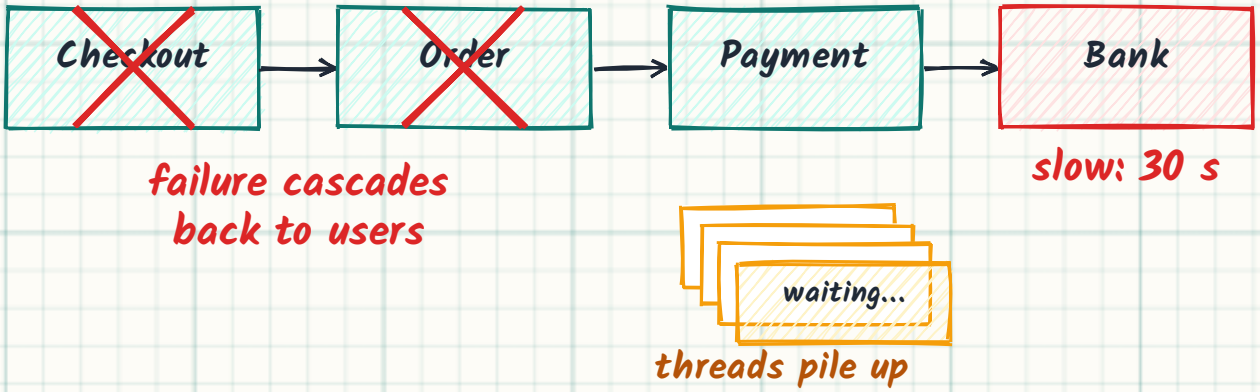
	REST	gRPC
Format	JSON (text)	Protobuf (binary)
Transport	HTTP/1.1 (usually)	HTTP/2 (multiplexing)
Contract	optional (OpenAPI)	required (.proto) + codegen
Browser	native	needs gRPC-Web / gateway
Streaming	limited	built in (4 kinds)
Debugging	easy (curl, Postman)	harder (binary)
Best for	public APIs, browser/mobile, CRUD	internal service-to-service, high volume, streaming

6 ShopNest: REST outside, gRPC inside



Common setup
Clients → **REST** → API Gateway → **gRPC** → internal services.

7 Why long sync chains are dangerous



- ✗ **Availability multiplies** down
- ✗ **Latency adds** up
- ✗ **Cascading failure**: one slow service fills threads upstream
- ✗ **Temporal coupling**: every service must be up at the same time

8 5 rules for sync calls

1. **Timeout** on every call, never wait forever
2. Keep chains **short** (2–3 hops)
3. **Retry** only idempotent calls, with exponential backoff
4. Add a **circuit breaker** (Lesson 14)
5. Caller doesn't need the answer now? **Publish an event** (Lesson 6)

> *Idempotent?*
> Safe to repeat with the same result: GET, PUT, DELETE. POST /payments is **not**: retrying it could charge twice, unless you use an idempotency key (Lesson 13).

? Interview Corner

Q. REST vs gRPC?

A. REST: JSON over HTTP, universal and easy to debug. gRPC: binary Protobuf over HTTP/2, strict contract, codegen, streaming. **REST outside, gRPC inside.**

Q. Why is gRPC faster?

A. Smaller binary messages, faster parsing, and HTTP/2 multiplexing many calls over one connection.

Q. Availability of a chain of 4 services at 99.9%?

A. Multiply: $0.999^4 \approx 99.6\%$. Every extra hop lowers it.

Q. What is a cascading failure?

A. One slow or failing service makes its callers wait until their threads run out, and the failure spreads upstream.

Q. How do you prevent it?

A. Timeouts, short chains, retries with backoff for idempotent calls, circuit breakers, and async events.

⚡ Revise in 60 seconds

- → Sync = **wait** for the answer
- → **REST**: resources, verbs, JSON
- → **gRPC**: .proto, Protobuf, HTTP/2
- → REST outside · gRPC inside
- → Chain availability **multiplies**
- → Latency **adds**
- → **Cascading failure**
- → Timeouts · short chains · events

Next lesson → MS-06
Asynchronous communication: **events, queues & event-driven architecture.**
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**